



Tarea #1

Compiladores e intérpretes

Gramática BNF - Lenguaje de programación Imperativo

Estudiantes:

Tayler Wynta Rodríguez | 2024143103

Maikel Flores Navarro | 2024148346

Profesor:

Ing. Allan Rodriguez Davila

II Semestre 2026

Índice

Descripción del problema	3
Diseño del programa	3
Símbolo inicial	3
Lista de no terminales	3
Lista de terminales	6
Producciones	8
1. Elementos léxicos y literales	8
2. Operadores	9
3. Delimitadores y símbolos.....	10
4. Tipos y palabras reservadas.....	10
5. Arreglos.....	11
6. Expresiones aritméticas	11
7. Expresiones relacionales.....	12
8. Expresiones lógicas	13
9. Expresión general.....	13
10. Variables y asignaciones.....	14
11. Funciones	14
12. Comentarios.....	14
13. Entrada y salida	15
14. Return y break	15
15. Estructura for.....	15
16. Estructura if-elif-else	15
17. Estructura while.....	16
18. Sentencias.....	16
19. Bloques	16
20. Función principal.....	16
21. Declaraciones globales	17
22. Estructura general del programa	17
Análisis de resultados	17
Lecciones aprendidas	17
Objetivos alcanzados	17
Objetivos no logrados / pendientes de validación semántica	18

Descripción del problema

Un grupo de desarrolladores necesita crear un nuevo lenguaje imperativo, ligero, orientado a realizar operaciones básicas de configuración de chips para sistemas empujados. Dado el crecimiento constante de esta industria y la necesidad de lenguajes cada vez más livianos y potentes para configurar dichos chips, el primer paso del proyecto consiste en diseñar una gramática simple y poderosa que defina formalmente la sintaxis de este lenguaje.

La gramática debe soportar, entre otros elementos: declaración de variables locales y globales con tipado explícito y fuerte (entero, flotante, booleano, carácter, cadena y arreglo estático unidimensional de enteros o flotantes); expresiones aritméticas, relacionales y lógicas con precedencia y asociatividad; estructuras de control if-elif-else, while y for; sentencias return y break; funciones con parámetros y valor de retorno; entrada/salida estándar mediante read y write; comentarios de una línea y de múltiples líneas; y un único punto de entrada del programa (procedimiento principal, de tipo void y sin parámetros). La gramática se desarrolla en notación BNF.

Diseño del programa

Símbolo inicial

El símbolo inicial de la gramática es:

<Programa>

Lista de no terminales

La gramática define 119 símbolos no terminales:

Símbolos
<Letra>
<Digito>
<Digito_No_Cero>
<Caracter>
<Id>
<Lit_Int>
<Parte_Entera>
<Parte_Decimal>
<Lit_Float>
<Lit_Bool>
<Caracter_String>

Símbolos
<String_Caracteres>
<Lit_String>
<Lit_Char>
<Op_Suma>
<Op_Resta>
<Op_Mult>
<Op_Div>
<Op_DivEnt>
<Op_Mod>
<Op_Pot>
<Op_Neg>
<Op_Inc>
<Op_Dec>
<Op_Menor>
<Op_Menor_Ig>
<Op_Mayor>
<Op_Mayor_Ig>
<Op_Igual>
<Op_Dif>
<Op_And>
<Op_Or>
<Op_Not>
<Op_Asig>
<Bloque_Ini>
<Bloque_Fin>
<Par_Ini>
<Par_Fin>
<Cor_Ini>
<Cor_Fin>
<Fin_Expr>
<Com_Linea>
<Com_Multi_Ini>
<Com_Multi_Fin>
<Coma>
<Tipo>
<Main>
<If>
<Elif>

Símbolos
<Else>
<While>
<For>
<Return>
<Break>
<Val>
<Read>
<Write>
<Void>
<Lista_Int>
<Lista_Float>
<Tamano_Arreglo>
<Arreglo_Int>
<Arreglo_Float>
<Arreglo>
<Indice>
<Acceso_Arreglo>
<Asignacion_Arreglo>
<Expr_Negativo>
<Expr_Incremento>
<Expr_Decremento>
<Operando>
<Expr_Potencia>
<Expr_Suma_Resta>
<Expr_Mult_Div>
<Expr_Aritmetica>
<Operando_Relacional>
<Operando_Igualdad>
<Expr_Relacional>
<Operando_Logico>
<Expr_Logica>
<Expr_Negacion_Logica>
<Expresion>
<Declaracion_Var_Simple>
<Declaracion_Var_Asignada>
<Asignacion_Var>
<Expr_Sin_Asignacion>
<Tipo_Retorno>

Símbolos
<Parametro>
<Parametros>
<Declaracion_Funcion>
<Argumento>
<Argumentos>
<Llamada_Funcion>
<Caracter_Comentario>
<Contenido_Comentario>
<Comentario_Linea>
<Comentario_Multi>
<Comentario>
<Sentencia_Read>
<Operando_Write>
<Sentencia_Write>
<Sentencia_Return>
<Sentencia_Break>
<Sentencia_For_Init>
<Asignacion_Var_Sin_Fin>
<Sentencia_For_Incremento>
<Sentencia_For>
<Sentencia_If>
<Sentencia_If_Resto>
<Sentencia_Elif_Lista>
<Sentencia_While>
<Sentencia>
<Bloque>
<Lista_Sentencias>
<Funcion_Main>
<Declaracion_Global>
<Elemento_Programa>
<Lista_Elementos>
<Programa>

Lista de terminales

La gramática define 59 símbolos terminales (palabras reservadas, operadores, delimitadores, literales y clases de caracteres):

Símbolos
"[a-zA-Z]"
"[0-9]"
"[1-9]"
"0"
"."
"true"
"false"
"[^"»¿?ε:əŋΓλθΣ i!]"
"ε"
"'"
"'"
"+"
"_"
"*"
"/"
"//"
"mod"
"pot"
"++"
"--"
"<"
"<="
">"
">="
"=="
"!="
"λ"
"θ"
"Σ"
"Γ"
"ζ."
":?"
"ε:"
":ə"
"f:"
"·λ"
"»"
" "

Símbolos
"."
"!"
"."
"int"
"float"
"bool"
"char"
"string"
"principal"
"if"
"elif"
"else"
"while"
"for"
"return"
"break"
"val"
"read"
"write"
"void"
"[^!]"

Producciones

A continuación, se listan todas las producciones de la gramática, agrupadas por categoría.

1. Elementos léxicos y literales

<Letra> ::= [a-zA-Z]

<Digito> ::= [0-9]

<Digito_No_Cero> ::= [1-9]

<Character> ::= <Letra> | <Digito>

<Id> ::= <Letra> <Character>*

<Lit_Int> ::= <Digito>+

<Parte_Entera> ::= "0" | <Digito_No_Cero> <Digito>*

<Parte_Decimal> ::= <Digito>* <Digito_No_Cero>

$\langle \text{Lit_Float} \rangle ::= \langle \text{Parte_Entera} \rangle "." \langle \text{Parte_Decimal} \rangle$
 $\langle \text{Lit_Bool} \rangle ::= "true" \mid "false"$
 $\langle \text{Caracter_String} \rangle ::= [^"'\rangle\langle ?\epsilon:\grave{\text{a}}\text{f}\lambda\theta\Sigma|;!]$
 $\langle \text{String_Caracteres} \rangle ::= \langle \text{Caracter_String} \rangle \langle \text{String_Caracteres} \rangle \mid \epsilon$
 $\langle \text{Lit_String} \rangle ::= "" \langle \text{String_Caracteres} \rangle ""$
 $\langle \text{Lit_Char} \rangle ::= "" \langle \text{Caracter_String} \rangle ""$

2. Operadores

2.1 Operadores aritméticos

$\langle \text{Op_Suma} \rangle ::= "+"$
 $\langle \text{Op_Resta} \rangle ::= "-"$
 $\langle \text{Op_Mult} \rangle ::= "*"$
 $\langle \text{Op_Div} \rangle ::= "/"$
 $\langle \text{Op_DivEnt} \rangle ::= "//"$
 $\langle \text{Op_Mod} \rangle ::= "mod"$
 $\langle \text{Op_Pot} \rangle ::= "pot"$
 $\langle \text{Op_Neg} \rangle ::= "-"$
 $\langle \text{Op_Inc} \rangle ::= "++"$
 $\langle \text{Op_Dec} \rangle ::= "--"$

2.2 Operadores relacionales

$\langle \text{Op_Menor} \rangle ::= "<"$
 $\langle \text{Op_Menor_Ig} \rangle ::= "<="$
 $\langle \text{Op_Mayor} \rangle ::= ">"$
 $\langle \text{Op_Mayor_Ig} \rangle ::= ">="$
 $\langle \text{Op_Igual} \rangle ::= "=="$
 $\langle \text{Op_Dif} \rangle ::= "!="$

2.3 Operadores lógicos

<Op_And> ::= "λ"

<Op_Or> ::= "θ"

<Op_Not> ::= "Σ"

2.4 Asignación

<Op_Asig> ::= "I"

3. Delimitadores y símbolos

<Bloque_Ini> ::= "¿:"

<Bloque_Fin> ::= ":?"

<Par_Ini> ::= "ε:"

<Par_Fin> ::= ":ε"

<Cor_Ini> ::= "f:"

<Cor_Fin> ::= ":f"

<Fin_Expr> ::= ">>"

<Com_Linea> ::= "|"

<Com_Multi_Ini> ::= "¡"

<Com_Multi_Fin> ::= "!"

<Coma> ::= ","

4. Tipos y palabras reservadas

<Tipo> ::= "int" | "float" | "bool" | "char" | "string"

<Main> ::= "principal"

<If> ::= "if"

<Elif> ::= "elif"

<Else> ::= "else"

<While> ::= "while"

<For> ::= "for"

<Return> ::= "return"

<Break> ::= "break"

<Val> ::= "val"
<Read> ::= "read"
<Write> ::= "write"
<Void> ::= "void"

5. Arreglos

<Lista_Int> ::= <Lit_Int>
<Lista_Int> ::= <Lit_Int> <Coma> <Lista_Int>
<Lista_Float> ::= <Lit_Float>
<Lista_Float> ::= <Lit_Float> <Coma> <Lista_Float>
<Tamano_Arreglo> ::= <Lit_Int>
<Arreglo_Int> ::= <Val> "int" <Id> <Cor_Ini> <Tamano_Arreglo> <Cor_Fin> <Fin_Expr>
<Arreglo_Int> ::= <Val> "int" <Id> <Cor_Ini> <Tamano_Arreglo> <Cor_Fin> <Op_Asig>
<Bloque_Ini> <Lista_Int> <Bloque_Fin> <Fin_Expr>
<Arreglo_Float> ::= <Val> "float" <Id> <Cor_Ini> <Tamano_Arreglo> <Cor_Fin> <Fin_Expr>
<Arreglo_Float> ::= <Val> "float" <Id> <Cor_Ini> <Tamano_Arreglo> <Cor_Fin> <Op_Asig>
<Bloque_Ini> <Lista_Float> <Bloque_Fin> <Fin_Expr>
<Arreglo> ::= <Arreglo_Int>
<Arreglo> ::= <Arreglo_Float>
<Indice> ::= <Lit_Int>
<Acceso_Arreglo> ::= <Id> <Cor_Ini> <Indice> <Cor_Fin>
<Asignacion_Arreglo> ::= <Acceso_Arreglo> <Op_Asig> <Expresion> <Fin_Expr>

6. Expresiones aritméticas

6.1 Expresiones unarias

<Expr_Negativo> ::= <Op_Neg> <Lit_Int>
<Expr_Negativo> ::= <Op_Neg> <Lit_Float>
<Expr_Incremento> ::= <Id> <Op_Inc>
<Expr_Decremento> ::= <Id> <Op_Dec>

6.2 Operandos aritméticos

$\langle \text{Operando} \rangle ::= \langle \text{Id} \rangle$

$\langle \text{Operando} \rangle ::= \langle \text{Lit_Int} \rangle$

$\langle \text{Operando} \rangle ::= \langle \text{Lit_Float} \rangle$

$\langle \text{Operando} \rangle ::= \langle \text{Acceso_Arreglo} \rangle$

$\langle \text{Operando} \rangle ::= \langle \text{Par_Ini} \rangle \langle \text{Expr_Aritmetica} \rangle \langle \text{Par_Fin} \rangle$

$\langle \text{Operando} \rangle ::= \langle \text{Expr_Negativo} \rangle$

$\langle \text{Operando} \rangle ::= \langle \text{Expr_Incremento} \rangle$

$\langle \text{Operando} \rangle ::= \langle \text{Expr_Decremento} \rangle$

$\langle \text{Operando} \rangle ::= \langle \text{Llamada_Funcion} \rangle$

6.3 Precedencia y asociatividad

$\langle \text{Expr_Potencia} \rangle ::= \langle \text{Operando} \rangle$

$\langle \text{Expr_Potencia} \rangle ::= \langle \text{Operando} \rangle \langle \text{Op_Pot} \rangle \langle \text{Expr_Potencia} \rangle$

$\langle \text{Expr_Suma_Resta} \rangle ::= \langle \text{Expr_Potencia} \rangle$

$\langle \text{Expr_Suma_Resta} \rangle ::= \langle \text{Expr_Suma_Resta} \rangle \langle \text{Op_Suma} \rangle \langle \text{Expr_Potencia} \rangle$

$\langle \text{Expr_Suma_Resta} \rangle ::= \langle \text{Expr_Suma_Resta} \rangle \langle \text{Op_Resta} \rangle \langle \text{Expr_Potencia} \rangle$

$\langle \text{Expr_Mult_Div} \rangle ::= \langle \text{Expr_Suma_Resta} \rangle$

$\langle \text{Expr_Mult_Div} \rangle ::= \langle \text{Expr_Mult_Div} \rangle \langle \text{Op_Mult} \rangle \langle \text{Expr_Suma_Resta} \rangle$

$\langle \text{Expr_Mult_Div} \rangle ::= \langle \text{Expr_Mult_Div} \rangle \langle \text{Op_Div} \rangle \langle \text{Expr_Suma_Resta} \rangle$

$\langle \text{Expr_Mult_Div} \rangle ::= \langle \text{Expr_Mult_Div} \rangle \langle \text{Op_DivEnt} \rangle \langle \text{Expr_Suma_Resta} \rangle$

$\langle \text{Expr_Aritmetica} \rangle ::= \langle \text{Expr_Mult_Div} \rangle$

$\langle \text{Expr_Aritmetica} \rangle ::= \langle \text{Expr_Aritmetica} \rangle \langle \text{Op_Mod} \rangle \langle \text{Expr_Mult_Div} \rangle$

7. Expresiones relacionales

$\langle \text{Operando_Relacional} \rangle ::= \langle \text{Expr_Aritmetica} \rangle$

$\langle \text{Operando_Igualdad} \rangle ::= \langle \text{Operando_Relacional} \rangle$

$\langle \text{Operando_Igualdad} \rangle ::= \langle \text{Lit_Bool} \rangle$

$\langle \text{Operando_Igualdad} \rangle ::= \langle \text{Lit_Char} \rangle$

$\langle \text{Expr_Relacional} \rangle ::= \langle \text{Cor_Ini} \rangle \langle \text{Operando_Relacional} \rangle \langle \text{Cor_Fin} \rangle \langle \text{Op_Menor} \rangle \langle \text{Cor_Ini} \rangle$
 $\langle \text{Operando_Relacional} \rangle \langle \text{Cor_Fin} \rangle$
 $\langle \text{Expr_Relacional} \rangle ::= \langle \text{Cor_Ini} \rangle \langle \text{Operando_Relacional} \rangle \langle \text{Cor_Fin} \rangle \langle \text{Op_Menor_Ig} \rangle$
 $\langle \text{Cor_Ini} \rangle \langle \text{Operando_Relacional} \rangle \langle \text{Cor_Fin} \rangle$
 $\langle \text{Expr_Relacional} \rangle ::= \langle \text{Cor_Ini} \rangle \langle \text{Operando_Relacional} \rangle \langle \text{Cor_Fin} \rangle \langle \text{Op_Mayor} \rangle \langle \text{Cor_Ini} \rangle$
 $\langle \text{Operando_Relacional} \rangle \langle \text{Cor_Fin} \rangle$
 $\langle \text{Expr_Relacional} \rangle ::= \langle \text{Cor_Ini} \rangle \langle \text{Operando_Relacional} \rangle \langle \text{Cor_Fin} \rangle \langle \text{Op_Mayor_Ig} \rangle$
 $\langle \text{Cor_Ini} \rangle \langle \text{Operando_Relacional} \rangle \langle \text{Cor_Fin} \rangle$
 $\langle \text{Expr_Relacional} \rangle ::= \langle \text{Cor_Ini} \rangle \langle \text{Operando_Igualdad} \rangle \langle \text{Cor_Fin} \rangle \langle \text{Op_Igual} \rangle \langle \text{Cor_Ini} \rangle$
 $\langle \text{Operando_Igualdad} \rangle \langle \text{Cor_Fin} \rangle$
 $\langle \text{Expr_Relacional} \rangle ::= \langle \text{Cor_Ini} \rangle \langle \text{Operando_Igualdad} \rangle \langle \text{Cor_Fin} \rangle \langle \text{Op_Dif} \rangle \langle \text{Cor_Ini} \rangle$
 $\langle \text{Operando_Igualdad} \rangle \langle \text{Cor_Fin} \rangle$

8. Expresiones lógicas

$\langle \text{Operando_Logico} \rangle ::= \langle \text{Lit_Bool} \rangle$
 $\langle \text{Operando_Logico} \rangle ::= \langle \text{Id} \rangle$
 $\langle \text{Operando_Logico} \rangle ::= \langle \text{Expr_Relacional} \rangle$
 $\langle \text{Operando_Logico} \rangle ::= \langle \text{Expr_Negacion_Logica} \rangle$
 $\langle \text{Operando_Logico} \rangle ::= \langle \text{Llamada_Funcion} \rangle$
 $\langle \text{Expr_Logica} \rangle ::= \langle \text{Par_Ini} \rangle \langle \text{Operando_Logico} \rangle \langle \text{Par_Fin} \rangle \langle \text{Op_And} \rangle \langle \text{Par_Ini} \rangle$
 $\langle \text{Operando_Logico} \rangle \langle \text{Par_Fin} \rangle$
 $\langle \text{Expr_Logica} \rangle ::= \langle \text{Par_Ini} \rangle \langle \text{Operando_Logico} \rangle \langle \text{Par_Fin} \rangle \langle \text{Op_Or} \rangle \langle \text{Par_Ini} \rangle$
 $\langle \text{Operando_Logico} \rangle \langle \text{Par_Fin} \rangle$
 $\langle \text{Expr_Negacion_Logica} \rangle ::= \langle \text{Op_Not} \rangle \langle \text{Par_Ini} \rangle \langle \text{Operando_Logico} \rangle \langle \text{Par_Fin} \rangle$
 $\langle \text{Expr_Negacion_Logica} \rangle ::= \langle \text{Op_Not} \rangle \langle \text{Par_Ini} \rangle \langle \text{Expr_Logica} \rangle \langle \text{Par_Fin} \rangle$

9. Expresión general

$\langle \text{Expresion} \rangle ::= \langle \text{Expr_Aritmetica} \rangle$
 $\langle \text{Expresion} \rangle ::= \langle \text{Expr_Relacional} \rangle$
 $\langle \text{Expresion} \rangle ::= \langle \text{Expr_Logica} \rangle$

<Expresion> ::= <Expr_Negacion_Logica>

<Expresion> ::= <Lit_Bool>

<Expresion> ::= <Lit_Char>

<Expresion> ::= <Lit_String>

10. Variables y asignaciones

<Declaracion_Var_Simple> ::= <Val> <Tipo> <Id> <Fin_Expr>

<Declaracion_Var_Asignada> ::= <Val> <Tipo> <Id> <Op_Asig> <Expresion> <Fin_Expr>

<Asignacion_Var> ::= <Id> <Op_Asig> <Expresion> <Fin_Expr>

<Expr_Sin_Asignacion> ::= <Expresion> <Fin_Expr>

11. Funciones

<Tipo_Retorno> ::= "int" | "float" | "char" | "bool"

<Parametro> ::= <Tipo> <Id>

<Parametros> ::= <Parametro>

<Parametros> ::= <Parametro> <Coma> <Parametros>

<Declaracion_Funcion> ::= <Tipo_Retorno> <Id> <Par_Ini> <Parametros> <Par_Fin> <Bloque>

<Argumento> ::= <Expresion>

<Argumentos> ::= <Argumento>

<Argumentos> ::= <Argumento> <Coma> <Argumentos>

<Llamada_Funcion> ::= <Id> <Par_Ini> <Argumentos> <Par_Fin>

12. Comentarios

<Caracter_Comentario> ::= [^!]

<Contenido_Comentario> ::= <Caracter_Comentario> <Contenido_Comentario> | ε

<Comentario_Linea> ::= <Com_Linea> <Contenido_Comentario>

<Comentario_Multi> ::= <Com_Multi_Ini> <Contenido_Comentario> <Com_Multi_Fin>

<Comentario> ::= <Comentario_Linea>

<Comentario> ::= <Comentario_Multi>

13. Entrada y salida

<Sentencia_Read> ::= <Read> <Par_Ini> <Id> <Par_Fin> <Fin_Expr>

<Operando_Write> ::= <Id>

<Operando_Write> ::= <Lit_Int>

<Operando_Write> ::= <Lit_Float>

<Operando_Write> ::= <Lit_Bool>

<Operando_Write> ::= <Lit_String>

<Sentencia_Write> ::= <Write> <Par_Ini> <Operando_Write> <Par_Fin> <Fin_Expr>

14. Return y break

<Sentencia_Return> ::= <Return> <Expresion> <Fin_Expr>

<Sentencia_Break> ::= <Break> <Fin_Expr>

15. Estructura for

<Sentencia_For_Init> ::= <Declaracion_Var_Asignada>

<Sentencia_For_Init> ::= <Asignacion_Var>

<Asignacion_Var_Sin_Fin> ::= <Id> <Op_Asig> <Expresion>

<Sentencia_For_Incremento> ::= <Expr_Incremento>

<Sentencia_For_Incremento> ::= <Expr_Decremento>

<Sentencia_For_Incremento> ::= <Asignacion_Var_Sin_Fin>

<Sentencia_For> ::= <For> <Par_Ini> <Sentencia_For_Init> <Expresion> <Fin_Expr>

<Sentencia_For_Incremento> <Par_Fin> <Bloque>

16. Estructura if-elif-else

<Sentencia_If> ::= <If> <Par_Ini> <Expresion> <Par_Fin> <Bloque> <Sentencia_If_Resto>

<Sentencia_If_Resto> ::= ϵ

<Sentencia_If_Resto> ::= <Sentencia_Elif_Lista>

<Sentencia_If_Resto> ::= <Else> <Bloque>

<Sentencia_If_Resto> ::= <Sentencia_Elif_Lista> <Else> <Bloque>

<Sentencia_Elif_Lista> ::= <Elif> <Par_Ini> <Expresion> <Par_Fin> <Bloque>

<Sentencia_Elif_Lista> ::= <Elif> <Par_Ini> <Expresion> <Par_Fin> <Bloque>
<Sentencia_Elif_Lista>

17. Estructura while

<Sentencia_While> ::= <While> <Par_Ini> <Expresion> <Par_Fin> <Bloque>

18. Sentencias

<Sentencia> ::= <Declaracion_Var_Simple>

<Sentencia> ::= <Declaracion_Var_Asignada>

<Sentencia> ::= <Asignacion_Var>

<Sentencia> ::= <Arreglo>

<Sentencia> ::= <Asignacion_Arreglo>

<Sentencia> ::= <Expr_Sin_Asignacion>

<Sentencia> ::= <Sentencia_Return>

<Sentencia> ::= <Sentencia_Break>

<Sentencia> ::= <Sentencia_Read>

<Sentencia> ::= <Sentencia_Write>

<Sentencia> ::= <Sentencia_If>

<Sentencia> ::= <Sentencia_While>

<Sentencia> ::= <Sentencia_For>

<Sentencia> ::= <Comentario>

19. Bloques

<Bloque> ::= <Bloque_Ini> <Lista_Sentencias> <Bloque_Fin>

<Lista_Sentencias> ::= <Sentencia>

<Lista_Sentencias> ::= <Sentencia> <Lista_Sentencias>

20. Función principal

<Funcion_Main> ::= <Void> <Main> <Par_Ini> <Par_Fin> <Bloque>

21. Declaraciones globales

<Declaracion_Global> ::= <Declaracion_Var_Simple>

<Declaracion_Global> ::= <Declaracion_Var_Asignada>

<Declaracion_Global> ::= <Arreglo>

22. Estructura general del programa

<Elemento_Programa> ::= <Comentario>

<Elemento_Programa> ::= <Declaracion_Global>

<Elemento_Programa> ::= <Declaracion_Funcion>

<Lista_Elementos> ::= <Elemento_Programa>

<Lista_Elementos> ::= <Elemento_Programa> <Lista_Elementos>

<Programa> ::= <Funcion_Main>

<Programa> ::= <Lista_Elementos> <Funcion_Main>

Análisis de resultados

Lecciones aprendidas

Aprendimos la estructura léxica que se utiliza en el diseño de un lenguaje, así como el manejo e interpretación de símbolos dentro de una gramática BNF. También reforzamos el flujo de trabajo en equipo, manteniendo comunicación constante con mi compañero durante todo el desarrollo.

Objetivos alcanzados

- Gramática BNF completa con símbolo inicial <Programa>, sin producciones genéricas de expresión, mediante niveles separados de precedencia: potencia, suma/resta, multiplicación/división/división entera y módulo.
- Declaración de variables locales y globales, con y sin asignación, usando la palabra reservada val y el símbolo de asignación =.
- Tipos int, float, bool, char y string, más arreglos estáticos de int y float: declaración, declaración con asignación (literal i: ... :?), acceso y modificación de elementos mediante índices j: ... j.

- Expresiones aritméticas binarias y unarias (negativo, ++, --), relacionales (con operandos entre f: :r) y lógicas (con operandos entre e: :e), incluyendo negación lógica.
- Estructuras de control if-elif-else, while y for, además de las sentencias return y break.
- Funciones con parámetros tipados y valor de retorno (int, float, char, bool), llamadas a función con argumentos, y procedimiento principal único (Funcion_Main), de tipo void y sin parámetros.
- Entrada estándar (read) y salida estándar (write), cada una con un único parámetro, según los tipos permitidos.
- Comentarios de una línea (!) y de múltiples líneas (¡ ... !).
- Delimitador de fin de sentencia (») aplicado de forma consistente en todas las sentencias.

Objetivos no logrados / pendientes de validación semántica

El diseño delega a la fase de análisis semántico (fuera de alcance) los siguientes aspectos, que no pueden expresarse mediante reglas puramente sintácticas:

- Verificación de tipado fuerte y explícito (por ejemplo, que mod y la base de pot reciban solo enteros, que pot use exponente entero positivo, o que los operandos de una expresión aritmética sean del mismo tipo).
- Restricción de que las condiciones de if, elif, while y for evalúen específicamente a un valor booleano, ya que sintácticamente aceptan cualquier <Expresion>.
- Verificación de que ++ y -- se apliquen únicamente a variables de tipo entero o flotante, y que el operador unario negativo se aplique solo a literales enteros o flotantes.
- Concordancia de tipos entre parámetros formales y argumentos reales en las llamadas a función, y entre el tipo de retorno declarado y el valor efectivamente retornado.